



Least Authority
PRIVACY MATTERS

CDK Data Availability
Security Audit Report

Polygon

Final Audit Report: 6 August 2024

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Issue A: Denial of Service Attack on Sync Service](#)

[Issue B: Race Condition Between FilterSequenceBatches and handleReorgs](#)

[Issue C: Unhandled Errors in consumeEvents Loop](#)

[Suggestions](#)

[Suggestion 1: Adopt a Verifiable Database / Batch Format](#)

[Suggestion 2: Use Libraries for Cryptography Instead of Inline Code](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Polygon has requested that Least Authority perform a security audit of CDK Data Availability. The cdk-data-availability project is a specialized Data Availability Node (DA Node) that is part of Polygon's CDK (Chain Development Kit) Validium.

Project Dates

- **April 15, 2024 - June 10, 2024:** Initial Code Review (*Completed*)
- **June 11, 2024:** Delivery of Initial Audit Report (*Completed*)
- **August 6, 2024:** Verification Review (*Completed*)
- **August 6, 2024:** Delivery of Final Audit Report (*Completed*)

Review Team

- George Gkitsas, Security Researcher and Engineer
- Xenofon Mitakidis, Security Researcher and Engineer
- Dominic Tarr, Security Researcher and Engineer

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of the CDK Data Availability followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repository is considered in scope for the review:

- CDK Data Availability:
<https://github.com/0xPolygon/cdk-data-availability>

Specifically, we examined the Git revision for our initial review:

- 38433c7d44a3f6e5d4407f36b44698ca0f4fd460

For the verification, we examined the Git revision:

- bd6eed4e9752c379dfdd873a0c6e31f044230301

For the review, this repository was cloned for use during the audit and for reference in this report:

- CDK Data Availability:
<https://github.com/LeastAuthority/Polygon-cdk-data-availability>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- Project README:
<https://github.com/0xPolygon/cdk-data-availability/blob/main/README.md>
- CDK website:
<https://polygon.technology/polygon-cdk>

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Whether requests are passed correctly to the network core;
- Key management, including secure private key storage and management of encryption and signing keys;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;
- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

Our team performed a security audit of the CDK Data Availability. The main function of the Data Availability Node (DA Node) is to replicate the data needed by the L2 network from the sequencer, both to make it available to clients and to have a replica in case of a Byzantine failure in the sequencer (that is, arbitrary incorrect behavior, malicious or otherwise). DA Nodes have very limited interactions with other computers. They request data from the Validium contract and batches from the sequencer, and sign batches proposed by the sequencer. They also provide batches to third parties ("availability").

We examined state-altering components, such as the synchronizer and database queries, for tampering data attack vectors leading to invalid states and could not identify any issues. However, we found that an attacker could perform a denial of service (DoS) attack ([Issue A](#)). Additionally, our team noted that in other locations in the codebase, the code is insufficiently defensive against network errors ([Issue C](#)), and a race condition could occur ([Issue B](#)), which is another Issue relating to distributed systems.

System Design

Our team found that security has generally been taken into consideration in the design of the CDK Data Availability. Given that there are minimal outside interactions (only three external APIs are exposed, and one of those can be called only by the sequencer) greatly limits the possibility for an attack.

However, our team found that the system relies heavily on Golang concurrency features. Channels are used to pass events between different stages of processing. As a result, our team identified one instance

where this concurrency could result in a race condition ([Issue B](#)). There were also several cases where the system assumes that a network request will always be successful, while it may actually fail and need to be retried later ([Issue C](#)).

Code Quality

We performed a manual review of the repositories in scope and found the codebases to be generally organized and well-written.

Tests

The repositories in scope contain extensive unit tests, including mocks as well as checks that verify whether errors are returned on invalid data. Our team found this to be sufficient for general application logic; however, more test coverage is needed to detect distributed system issues relating to network unreliability or timings.

Documentation and Code Comments

The documentation available describes the CDK system in broad strokes but does not precisely describe the Data Availability Node, which was being audited. In some instances in the codebase, comments convey useful information; however, in several other instances, the comments are redundant and only repeat function names and keywords (e.g., [here](#)). We recommend using godoc or pkgsite for enhanced structure.

Scope

The data availability node constitutes only a small part of the Polygon CDK system. In some instances during the review, it was necessary to investigate the sequencer, Validium, and data committee contracts, which were out of the scope of this audit, in order to properly understand the behavior of the DA node. We recommend performing a comprehensive, follow-up security audit of the aforementioned components once the Issues and suggestions in this report have been adequately addressed.

Dependencies

Our team did not identify any security concerns resulting from the unsafe use of dependencies.

Specific Issues & Suggestions

We list the issues and suggestions found during the review, in the order we reported them. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	STATUS
Issue A: Denial of Service Attack on Sync Service	Resolved
Issue B: Race Condition Between FilterSequenceBatches and handleReorgs	Resolved
Issue C: Unhandled Errors in consumeEvents Loop	Resolved
Suggestion 1: Adopt a Verifiable Database / Batch Format	Partially Resolved
Suggestion 2: Use Libraries for Cryptography Instead of Inline Code	Resolved

Issue A: Denial of Service Attack on Sync Service

Location

services/sync/sync.go#L43-L58

Synopsis

An attacker can request a large amount of data from the publicly accessible sync service, which would cause the Data Availability Node to allocate a significant amount of memory that will only be released slowly.

Impact

If an attacker performs many of these attacks, they could cause fatal out-of-memory errors, thus causing the Data Availability (DA) Nodes to crash at a relatively low cost to themselves. This could disrupt service to users of the system and possibly make consensus slower.

Preconditions

The attacker would have to know the IP address of a data availability node. Note that in case of large batches, it is easier for the attacker to exploit this Issue.

Feasibility

Highly Feasible.

Technical Details

Using the JSONRpc sync service, an attacker can request a large number of batches in a single request, causing the DA node to use up memory. For example, if the attacker requests 1000 blocks that are known to be 1MB each, the DA node will need to use 1GB of memory to address that request. The DA node will subsequently collect all data requested, marshal it to JSON format, then send it to the HTTP response. If the attacker receives that data only slowly, it will remain in the DA node until the entire response is sent. The attacker is limited only by the maximum size allowed for the HTTP headers. In the GoLang HTTP library, this is 1MB by default, which would be enough to request 16k blocks at once. It is likely that the attacker would only need to make 10s or less of such requests to be able to cause a fatal out-of-memory error, which would cause the node to crash.

Mitigation

We suggest setting a reasonable limit to the number of blocks that can be requested at once and/or limiting the rate that sync requests can be made to significantly reduce the impact of the attack. Note that making a large request would still allocate more resources on the DA node than on the attacker.

Remediation

To fully remediate the attack, we recommend limiting requests (by size and rate), such that each request does not allocate more than a fixed amount per client, and one request for that client is fully handled before the next request is started. Our team acknowledges that denial-of-service attacks can always be attempted; however, it should not be trivial to perform such attacks with only a few resources being required.

Status

The Polygon team has added a rate limit in the service configuration, which applies to the HTTP server. The team has additionally added a limit to the list of hashes that could be provided to ListOffchainData endpoint (the maximum number allowed is 100). However, our team noted that the default setting (500 requests per second per ip) for the http rate limit is too high. While having a limit

does make it harder to perform the attack, it seems likely that 500 requests per second would still be enough for an attack to be carried out successfully. We recommend that operators configure `MaxRequestsPerIPAndSecond` to a much lower value, such as 50.

Verification

Resolved.

Issue B: Race Condition Between `FilterSequenceBatches` and `handleReorgs`

Location

[main/synchronizer/batches.go#L170-L187](#)

[main/synchronizer/batches.go#L116-L125](#)

Synopsis

Items retrieved from `FilterSequenceBatches` are emitted to the `bs.events` channel. The start block is then advanced, as seen in [main/synchronizer/batches.go#L170-L187](#). `handleReorgs` also interacts with the start block and runs concurrently to `filterEvents`, as seen in [main/synchronizer/batches.go#L116-L125](#).

Impact

During the mining process, network delays may sometimes occur. As a result, two blocks are mined simultaneously, and then another chain (different from the first one) wins. In other words, from the user's perspective, the longest chain changes. In the code, this is referred to as a "reorg." If a reorg is detected and handled while `FilterSequenceBatches` is running, this may cause the DA to have a gap, as some batches would not be downloaded. The sequencer and the DA committee would thus be the holders of the state. Additionally, if any batches are lost, it will not be possible to recreate the state of the system.

Preconditions

A DA node must have recently started or restarted and nearly finished loading batches with `FilterSequenceBatches`, when a reorg was detected.

Feasibility

Even though the Issue is highly dependent on the particular timing of the preconditions, it will eventually occur.

Technical Details

Each DA node regularly uses `FilterSequenceBatches` to download batches, saving the number of the last batch in the database using `setStartBlock` in groups of up to 64 batches by default (controlled by the `blockSizeBatch` configuration setting) or however many batches are currently available. As noted above, reorgs may sometimes occur during the mining process when the longest chain changes due to network delays. Reorgs are detected by polling for the latest block every second (not configurable). A reorg is detected when the current block number +1 is greater than or equal to the new block's number. When a reorg is detected, if the number of the reorg block is lower than the value given by `getStartBlock`, the start block is rewound and the synchronizer downloads batches from that point the next time it runs `FilterSequenceBatches`. However, if the start block is lower than the reorg number, then no action is performed. Since `setStartBatch` is not called until *after* each group of (up to 64) batches have been processed, it is possible that `FilterSequenceBatches` would have started but not yet finished. In this case, `setStartBlock` would not be updated yet. When the reorg is subsequently detected, `getStartBlock` returns the original, low value without changing it. However,

FilterSequenceBatches then completes and updates setStartBlock to a value higher than the reorg. If FilterSequenceBatches had been a little faster, or the reorg had been detected a little later, then the reorg would have been handled. Consequently, as this was not the case, the reorg would be ignored, which may result in some inconsistent batches.

Remediation

We recommend ensuring that the reorg detector is aware of the activity of the synchronizer. More specifically, when batches are being retrieved, we recommend making the detector wait until each group is completed before processing the next.

Status

The Polygon team is currently using `sync.Mutex` to ensure that reorgs and FilterSequenceBatches do not run at the same time.

Verification

Resolved.

Issue C: Unhandled Errors in consumeEvents Loop

Location

[main/synchronizer/batches.go#L199-L201](#)

Synopsis

Events representing transactions to the L1 contract are processed in the `consumeEvents` loop. However, since errors are only logged, this could result in a gap in the database.

Impact

This Issue could result in a DA node having missing data.

Preconditions

This Issue can occur if `client.getTx` returns an error. This could happen at any time but is more likely in times of network congestion.

Feasibility

If the preconditions are met, this Issue will occur eventually.

Technical Details

The synchronizer contains several concurrent loops that communicate with each other using Go channels. `produceEvents` uses `FilterSequenceBatches` to request a series of transactions then writes them to a channel that feeds into `consumeEvents`. After writing a group of batches to that channel, `FilterSequenceBatches` updates the start block, such that the next time around the loop it can retrieve the next group of batches. If there is an error during one run, it will not update the start block. Hence, after a delay, it will retry that same group. As a result, intermittent failures will be retried and will work eventually. However, the loop inside `consumeEvents` does not handle intermittent failures as such. `consumeEvents` pulls an event from the channel and then calls `handleEvent`, which calls `client.getTx` (i.e., a `jsonrpc` HTTP method that may have an intermittent failure). If that fails, the loop simply logs a message and continues. Consequently, that transaction will not be retried; instead, it will move on to the next event, which might succeed, etc.

Remediation

We recommend moving `handleEvents` inside `FilterSequenceBatches`, such that if there is an error, it will not update the start batch and will retry it after a delay, to avoid having gaps.

Status

The Polygon team has moved `handleEvents` into `filterEvents`, such that events are handled before the `startBlock` is set. This shift causes `filterEvents` to be called again after a delay, which results in those requests being retried.

Verification

Resolved.

Suggestions

Suggestion 1: Adopt a Verifiable Database / Batch Format

Synopsis

In the current format, it is not possible to know that a given DA node has the complete data replica, as would be necessary in the case of a Byzantine failure of the sequencer.

Mitigation

Currently, for the DA node to have consistent data, it should either have recently downloaded it or not show any errors logged in the output. Additionally, [Issue B](#), or any other unknown issues, should not occur. The operators of DA nodes could compare lists of hashes (although it is possible and it is not a problem to have extra hashes); however, the code audited has no features to assist with this.

Remediation

If the data format of the batches change, such that each encoded batch begins with a pointer to the previous batch, then it would be possible to, from the latest batch, check whether a node also has the previous hash. Following such hashes all the way to the genesis batch for the protocol would confirm that the entire database is both valid and complete. When a batch is received via `SignSequence`, it must be checked that it points back at a valid state, verifying that it is a valid extension. The valid state of the database and top hash should be included in the status service so that it is possible for DA nodes to check that the entire committee has valid and complete data.

Status

The Polygon team has implemented some mitigations, which are likely to improve data consistency. A block number is now saved alongside each block, and there is also another process to check and heal gaps. If gaps are detected, they are reported in the status service. However, the recommended remediation of using a hash chain was not implemented. This means that the completeness of the database depends on the lack of bugs in the implementation, and thus cannot be cryptographically verified. The Polygon team stated that they did not adopt the suggested design to prioritize availability.

Verification

Partially Resolved.

Suggestion 2: Use Libraries for Cryptography Instead of Inline Code

Location

[main/types/sequence.go#L42-L64](#)

Synopsis

Cryptographic code should be used from libraries, not copied directly into the project code. Cryptographic code (e.g., hashes or signatures) is complex and specialized. A high degree of expertise and skills are required for making any changes to it. As a result, if it is added alongside the regular code, it might get unintentionally changed.

Mitigation

We recommend utilizing well-known libraries instead of copying code directly into the project.

Status

The Polygon team reported that they did not implement the suggestion because there was no suitable library for them to utilize. Our team thus performed further investigations and noted that the purpose of the custom code mentioned in this suggestion is to normalize signatures to address signature malleability. We found that the standard [Ethereum library](#), which the team is already using, does perform signature normalization by default. This was verified by reading library code, removing the custom normalization code, and running the tests (which did not fail). We suggested removing the custom normalization code, except for the addition of 27 to the recovery byte / chain ID in the byte index 64 of the signature, and the cdk-data-availability team implemented our new suggestion.

Verification

Resolved

About Least Authority

We believe that people have a fundamental right to privacy and that the use of secure solutions enables people to more freely use the Internet and other connected technologies. We provide security consulting services to help others make their solutions more resistant to unauthorized access to data and unintended manipulation of the system. We support teams from the design phase through the production launch and after.

The Least Authority team has skills for reviewing code in multiple Languages, such as C, C++, Python, Haskell, Rust, Node.js, Solidity, Go, JavaScript, ZoKrates, and circom, for common security vulnerabilities and specific attack vectors. The team has reviewed implementations of cryptographic protocols and distributed system architecture in cryptocurrency, blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, the team can utilize various tools to scan code and networks and build custom tools as necessary.

Least Authority was formed in 2011 to create and further empower freedom-compatible technologies. We moved the company to Berlin in 2016 and continue to expand our efforts. We are an international team that believes we can have a significant impact on the world by being transparent and open about the work we do.

For more information about our security consulting, please visit <https://leastauthority.com/security-consulting/>.

Our Methodology

We like to work with a transparent process and make our reviews a collaborative effort. The goals of our security audits are to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Manual Code Review

In manually reviewing all of the code, we look for any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's website to get a high level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. As we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation. We hypothesize what vulnerabilities may be present and possibly resulting in Issue entries, then for each, we follow the following Issue Investigation and Remediation process.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even before having verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyze the feasibility of an attack in a live system.

Suggested Solutions

We search for immediate and comprehensive mitigations that live deployments can take, and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our Initial Audit Report, and before we perform a verification review.

Before our report, including any details about our findings and the solutions are shared, we like to work with your team to find reasonable outcomes that can be addressed as soon as possible without an overly negative impact on pre-existing plans. Although the handling of issues must be done on a case-by-case basis, we always like to agree on a timeline for a resolution that balances the impact on the users and the needs of your project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess that the issues and suggestions are sufficiently addressed. When this analysis is completed, we update the report and provide a Final Audit Report that can be published in whole. If there are critical unaddressed issues, we suggest the report not be published and the users and other stakeholders be alerted of the impact. We encourage that all findings be dealt with and the Final Audit Report be shared publicly for the transparency of efforts and the advancement of security learnings within the industry.